

Written Report

Q1 Baseline and scope

SQLiFuzz collects HTTP requests through a reverse proxy, observes corresponding SQL queries and database responses through a database proxy, and fuzzes request parameters using database feedback. I am testing its special-character mutation strategy and syntax-error detection oracle.

- Paper citation: I Putu Arya Dharmaadi, Van-Thuan Pham, Fadi Mohsen, and Fatih Turkmen. 2026. SQLiFuzz: Uncovering SQL Injection in Any Web Applications. Proc. ACM Softw. Eng. 3, FSE, Article FSE142 (July 2026). <https://doi.org/10.1145/3808149>
- Relevant Sections: §3.4.2, “Mutation,” and §3.4.3, “Oracle Verification,” on page FSE142:9. Section 3.4.2 describes inserting or replacing parameter values with special characters intended to cause SQL parsing errors. Section 3.4.3 marks a request as successfully exploiting SQL injection when the mutated value appears in the corresponding SQL query and the database returns a syntax error.
- Authors' Artifact URL and Version: Repository URL: <https://github.com/websecfuzz/SQLiFuzz>, SQLiFuzz commit: [1b7e2ede42af8e2b9d6d1c44358851e9d4616db9](https://github.com/websecfuzz/SQLiFuzz/commit/1b7e2ede42af8e2b9d6d1c44358851e9d4616db9)
- Configuration: DVWA with PHP/MySQL and SQLiFuzz's database observer. Created php files for the matched control and primary case. Query tested: `SELECT first_name,last_name FROM users WHERE user_id = $id`. In `SQLiFuzz/sqlifuzz/sql_analysis.py`, I replaced both occurrences of `testloc = "/projects/fuzzing/SQLiFuzz/shared-data"` with: `testloc = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "shared-data"))` because data was not saving correctly with the hardcoded path.
- A1 Starting Point: In A1, I reproduced the DVWA result in Table 4, row A1, detecting both `sql_i` and `sql_i_blind` vulnerabilities on the site using SQLiFuzz and the custom crawler.

This case is within the paper's claimed scope because it uses a standard HTTP GET parameter and a MySQL query, which are part of SQLiFuzz's described operating scope. I test the special-character mutation strategy and syntax-error oracle described in Sections 3.4.2–3.4.3. The constructed filter accepts Boolean expressions that alter query results but rejects syntax-breaking inputs. The paper does not explicitly discuss this particular case.

Q2 Expected behavior

My primary failed case is [a2_filtered.php](#). The security property is that a user-supplied identifier should be treated as data and should not control the logical structure of the database query. The page instead concatenates the `id` parameter into a numeric SQL expression: `SELECT first_name,last_name FROM users WHERE user_id = $id`. The filter accepts digits and restricted Boolean expressions involving AND or OR, while rejecting other characters. Therefore, the filter blocks many malformed inputs but does not prevent SQL injection.

I established the vulnerability independently of SQLiFuzz by submitting these inputs directly to the [a2_filtered.php](#) page (screenshots in [/notes/ground_truth_boolean_check.md](#)):

- [/a2_filtered.php?id=1+AND+1%3D1&Submit=Submit](#) → ROWS=1
- [/a2_filtered.php?id=1+AND+1%3D0&Submit=Submit](#) → ROWS=0

Both expressions pass the filter and produce valid SQL. Their different results demonstrate that the input can introduce an additional SQL condition. Blocking apostrophes does not prevent this vulnerability.

My evidence consists of the filter and query construction in [/cases/a2_filtered.php](#) (line 28), together with the direct-request responses saved in [/notes/ground_truth_boolean_check.md](#). The expected security result is that the page is vulnerable; my testing hypothesis was that SQLiFuzz would miss it. SQLiFuzz's output alone cannot establish correctness because "no finding" could mean either that the page is safe or that its search and detection oracle failed to expose an existing vulnerability.

Q3 Observed result

[/raw/primary_case/filtered-run2](#) walkthrough:

First, I started the proxy listener with this command:

```
mitmdump --mode reverse:http://localhost:8081 \  
  --listen-port 8888 \  
  -s sqlifuzz/mitmproxy_addon.py \  
  2>&1 | tee "$trial_dir/terminal.log"
```

Then, I opened the http://localhost:8888/a2_filtered.php page in my browser and input the number 1 in the textbox. I submitted the input and its terminal output was recorded to [/raw/primary_case/filtered-run2/terminal.log](#) (line 52). Lines 113-120 show an HTTP 200 response containing INPUT REJECTED, followed by an associated database record containing USE db in [/raw/primary_case/filtered-run2/mysql_proxy_*.log](#) (line 545) rather than the vulnerable SELECT. Thus, the request reached the application but that payload was rejected before the target query. In the completed run, SQLiFuzz reported 70 fuzzing attempts and [No SQL Injection Detected](#) located in

[/raw/primary_case/filtered-run2/FR-dvwa--emmalee-VMware-Virtual-Platform-1790976014.txt](#).

This means the filtered cases completed 70 fuzzing attempts without reporting an injection. (The [fuzzed_req](#) count includes attempts against other parameters, including headers. It does not mean 70 unique payloads or 70 mutations of id.) However, it does not mean that the page is safe. Each case was tested in three runs. SQLiFuzz detected the control in 3/3 runs, with two fuzzing attempts per run. It detected neither the filtered GET case nor the POST variant in their three runs, each recording 70 fuzzing attempts. These are counts across repeated runs, not a detection rate across different vulnerabilities.

A potentially misleading result is a zero SQLi count. It describes the detector's findings, not the target's security. I checked its meaning against the independently verified Boolean injection.

*Terminal and database timestamps differ by four hours; see [/environment/environment.md](#). Database-log entries are identified using the database timestamps quoted in each run's terminal output.

**POST variant run 1 completed fuzzing and produced a report, but its reported 19-hour duration is unreliable. I have kept its detection outcome and evidence but have disclosed that duration for that trial should be excluded from runtime comparisons.

Q4 Controlled comparison

The primary case and matched control use the same authentication requirement, GET form, starting input [id=1](#), input-length restriction, database connection, query construction, and response format. They were tested using the same baseline installation and procedure. The file [/cases/difference_of_control_and_filtered.txt](#) shows the only code difference between [a2_control.php](#) and [a2_filtered.php](#).

The enabled filter rejects malformed inputs before executing the vulnerable SELECT. The disabled filter allows those inputs to reach the database. The [/derived/results_table.csv](#) was created using [/scripts/derive_results.py](#). Here is a truncated version of the table:

Case	Trials detecting the endpoint	Attempts per trial
Unfiltered GET control	3/3	2
Filtered GET primary case	0/3	70
Filtered POST variant	0/3	70

Each control report contains a matched [id=1'](#) payload and MySQL syntax error 1064. Each filtered report contains zero findings after 70 attempts. The control demonstrates that the shared installation can deliver a request, observe a corresponding SQL syntax error, and report

a SQLiFuzz finding. This weakens a general setup failure explanation. The filtered run shows the original SELECT being matched and fuzzing beginning, which rules out that the primary request never reached analysis.

My additional variant is [a2_filtered_post.php](#). Its difference from [a2_filtered.php](#) is that the form method is POST rather than GET. The filter and unsafe query are the same, additional differences are comments and whitespace in the actual code ([/cases/difference_of_filtered_and_filtered_post.txt](#))

My prediction recorded before running the variant was that SQLiFuzz would detect zero of the POST trials because the filter would reject the special-characters independently of the HTTP method. The observed outcome was correct.

*The prediction note is dated October 3, 2026 at 1:45pm EST, while the first variant log records fuzz completion on October 2, 18:25:06 EST and delayed final reporting on October 3 13:53:06 EST. To prove the time jump, journal entries show an advance of approximately 59 seconds in monotonic time but 19 hours 28 minutes in wall-clock time, ending with “Clock change detected.” Therefore, the first POST trial’s wall-clock duration is unreliable.

```
[14223.024806] emmalee-VMware-Virtual-Platform CRON[60077]: pam_unix(cron:session): session closed for user root
[14282.234818] emmalee-VMware-Virtual-Platform systemd-resolved[622]: Clock change detected. Flushing caches.

2026-10-02T18:25:01.143154-04:00 emmalee-VMware-Virtual-Platform CRON[60077]: pam_unix(cron:session): session closed for user root
2026-10-03T13:52:58.736149-04:00 emmalee-VMware-Virtual-Platform systemd-resolved[622]: Clock change detected. Flushing caches.
```

Q5 Failure family and cause

My results suggest a detection gap for numeric SQL injection points that accept valid Boolean expressions but reject the malformed-query probes used by SQLiFuzz. The smallest shared condition is unsafe concatenation into a numeric SQL expression combined with a filter that allows logical conditions while blocking syntax-breaking inputs. Both the GET primary case and POST variant have this condition.

The explanation connects to Sections 3.4.2 and 3.4.3 of the paper and to [test_SQLi](#) in [SQLiFuzz/sqlifuzz/SQLIFuzz.py](#). Its mutation helper prioritizes apostrophes and asterisks before random special characters, its loop permits 70 attempts, and its detection branch checks for a matched query with a SQL syntax error.

For the primary case, the terminal record directly shows an apostrophe mutation receiving INPUT REJECTED. The PHP source explains why that input cannot reach the vulnerable SELECT. Meanwhile, the independently supplied Boolean expressions reach valid queries and change results without requiring a syntax error.

A plausible alternative explanation is poor parameter prioritization. The primary log shows that the first mutation targets Sec-Fetch-User, so some budget is spent outside the injection point. However, later mutations explicitly target [id](#) and are rejected. This weakens an explanation based on never testing the vulnerable parameter.

The observed pattern is a repeatable miss within the stated budget for the two filtered cases, alongside successful detection of the control. The source and request records support

filter rejection, but do not prove that every mutation was rejected or that every similar vulnerability is undetectable.

Q6 A decision made during the investigation

A nontrivial decision was to move from investigating the viewport failure to investigating SQLiFuzz's mutation strategy and detection oracle. In [/notes/investigation_notes.md](#), on October 1, 2026, I found a 502 Bad Gateway page in the screenshot associated with the viewport error from Assignment 1. That observation weakened the initial explanation that the crawler could not locate the application button. It suggested that application or proxy availability could be involved, without establishing the exact cause.

On October 2, 2026, I moved to manually supplied requests and constructed PHP cases. My [/notes/decisions.md](#) documents this change. I kept this direction because the matched experiment produced clearer results and the testing was quicker taking only 3-4 minutes per trial rather than 20-25 minutes.

One remaining uncertainty is how much irrelevant parameter selection contributes to the observed search behavior. An additional experiment that restricts mutation to `id`, while preserving the same filter and budget, would isolate that contribution. My current evidence cannot establish the limitation's frequency across real applications or a universal inability to detect Boolean injection.

Project proposal

Q7 Proposed heuristic or tool change

For A3, I propose adding database-informed paired Boolean probes for supported numeric injection contexts.

The input is an HTTP request, a candidate numeric parameter, and its associated SQL query. The new strategy uses the observed query context to decide whether the parameter occupies an unquoted numeric expression. For example, it generates paired valid expressions:

- Original: `id =1`
- True: `id=1 AND 1=1`
- False: `id=1 AND 1=0`

The decision rule would be:

- Identify a supported numeric parameter/query context.
- Send a true/false pair.
- Associate each request with its SQL query and database outcome.
- Check whether the injected Boolean operators became SQL structures.
- Repeat the pair to check results stability.
- Report an evidence-backed finding, or mark the result inconclusive.

For my primary case, both probes should pass the filter. The true expression should return one row and the false expression zero rows. The implementation will integrate with `test_SQLi` in `SQLiFuzz/sqlifuzz/SQLIFuzz.py` and the existing query-observation helpers. It needs an additional detection path that accepts supported evidence of logical query manipulation without requiring a syntax error.

Initially, I will compare stable HTTP responses, such as `ROWS=1` versus `ROWS=0`, while using the database observer to verify that the supplied Boolean expression became executable SQL logic. A response difference alone will not establish injection.

The output will include the paired requests, corresponding SQL queries, observed result difference, and confirmed or inconclusive results. My proposed contribution is a focused integration with SQLiFuzz's database visibility. The change should help the current filtered GET and POST cases. It may still miss quoted-string, transformed-input, or stateful contexts. Dynamic responses or incorrect query association could produce misleading differences.

Q8 Project scope and first evaluation

Research Question: Can database-informed paired Boolean probing improve SQLiFuzz's detection of filtered numeric injection points without introducing findings on matched safe cases?

I will compare the recorded baseline with the modified implementation under the same target state and request budget. Metrics will be independently confirmed vulnerable endpoints detected and false findings on safe endpoints. I will also report inconclusive outcomes, unsuccessful runs, and requests used before detection.

Development cases will include the current filtered GET page, filtered POST page, unfiltered control, and a new safe parameterized-query control. For additional evaluation, I will reserve a separately defined set of local cases before tuning. These will vary numeric query contexts and filter restrictions and include both vulnerable and safe implementations. Their security labels will be established through source inspection and direct execution checks independently of the baseline and modified detector. I will report this as evaluation on constructed cases, without claiming real-world generality.

The existing unfiltered control will provide a regression check: the modified tool should retain its successful detection. A parameterized-query control will test whether the new oracle mistakenly treats SQL-like input data as executable logic. Initial cost measurements will include requests sent and elapsed time, measured separately for active fuzzing and total run duration. A later scaling experiment will increase the number of candidate parameters per request while holding the query workload and database contents fixed.

A running prototype requires numeric-context recognition, paired-payload generation, request replay, query association, a result-comparison oracle, evidence reporting, and a comparison. Baseline and modified runs will use separate output directories and recorded code versions.

The main risk is unreliable context recognition or attribution of response differences to injection. My fallback is a prototype supporting only a clearly defined numeric WHERE-clause pattern with stable responses; other contexts will be marked unsupported or inconclusive. Assignment 3 will be a solo contribution, therefore no work needs to be distributed.

References

Dharmaadi, I. P. A., Pham, V.-T., Mohsen, F., & Turkmen, F. (2026, June 30). *SQLiFuzz: Uncovering SQL Injection in Any Web Applications*. ACM Digital Library. Retrieved September 13, 2026, from <https://dl.acm.org/doi/pdf/10.1145/3808149>